

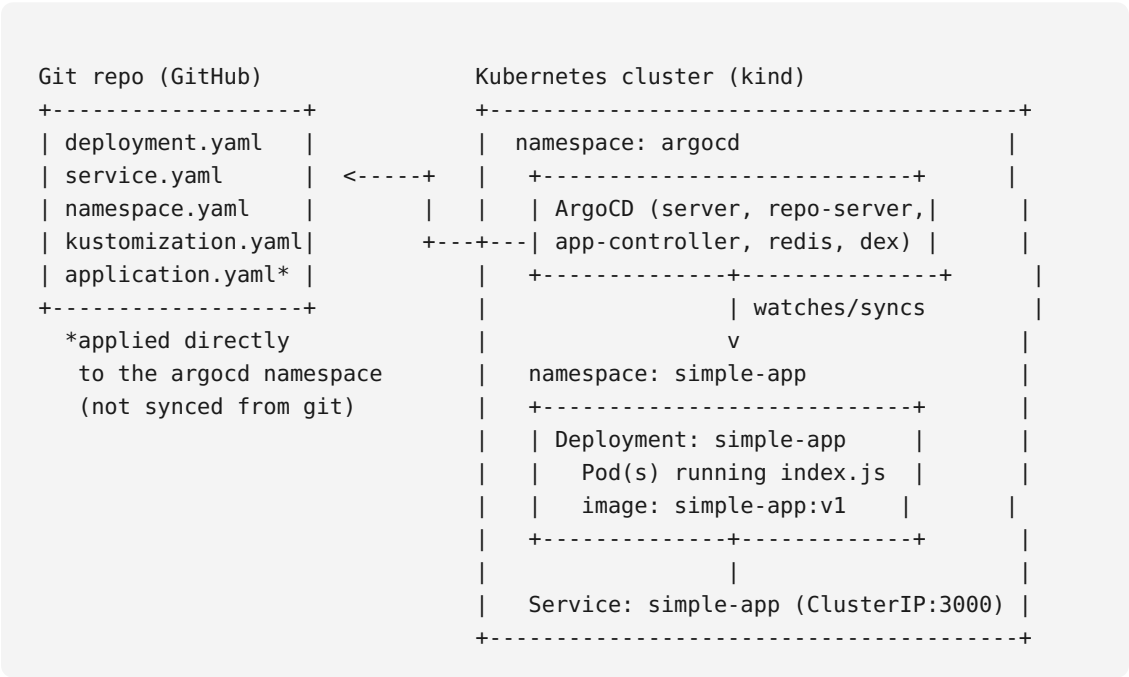
simple-app — ArgoCD Demo: Setup Report

Repo: omar-ait-bougnsa/simple-app | Cluster: kind (simple-app-cluster)

1. What the app is

A minimal Node.js/Express server (`index.js`) that responds with "hello from argo CD" on GET `/` , listening on port 3000. It's packaged as a Docker image and deployed to Kubernetes, with ArgoCD managing the deployment by continuously syncing manifests from this Git repository.

2. How it all fits together



3. Files and what each one does

File	Purpose	Status
<code>Dockerfile</code>	Builds the app image (node:18-alpine, npm install, runs index.js)	committed & pushed
<code>index.js</code> / <code>package.json</code>	The Express app itself	committed & pushed
<code>deployment.yaml</code>	K8s Deployment: 2 replicas of the app, image <code>simple-app:v1</code> , <code>imagePullPolicy: Never</code>	committed & pushed
<code>service.yaml</code>	K8s Service exposing the Deployment's port 3000 inside the cluster (ClusterIP)	written, not yet committed
<code>namespace.yaml</code>		

	Defines the <code>simple-app</code> namespace the app lives in	written, not yet committed
<code>kustomization.yaml</code>	Tells ArgoCD to build this folder with Kustomize, listing <code>deployment.yaml</code> + <code>service.yaml</code> as resources	written, not yet committed
<code>application.yaml</code>	The ArgoCD <code>Application</code> CRD: tells ArgoCD which repo/branch/path to watch and which cluster/namespace to deploy to. Applied directly to the <code>argocd</code> namespace (does not live in the synced path itself)	applied to cluster, repo not reachable yet

4. Current live status (checked just now)

Check	Command	Result
ArgoCD pods	<code>kubectl get pods -n argocd</code>	All Running
ArgoCD Application	<code>kubectl get application simple-app -n argocd</code>	SYNC STATUS: Unknown
Root cause	<code>kubectl get application simple-app -n argocd -o jsonpath='{.status.conditions}'</code>	"Failed to load target state... SSH agent requested but SSH_AUTH_SOCK not-specified"
App namespace	<code>kubectl get ns simple-app</code>	Not found (never synced)
Docker image	<code>docker images grep simple-app</code>	simple-app:v1 built locally
Image inside kind cluster	<code>kind load docker-image ...</code>	Not loaded yet
GitHub repo reachability	<code>curl https://github.com/omar-ait-bougnsa/simple-app</code>	404 — repo is private

5. Why it isn't fully working yet

Two independent blockers, both confirmed live in the cluster:

1. **ArgoCD can't clone the repo.** Your `application.yaml` points at the SSH URL `git@github-omar-ait-bougnsa:omar-ait-bougnsa/simple-app.git`. That works from your laptop because your own SSH key/agent is configured locally — but ArgoCD's `repo-server` pod runs inside the cluster with no SSH key and no SSH agent, so it fails immediately. Since the repo is also private, switching to a plain HTTPS URL would *also* fail without credentials.

2. **The image isn't inside the kind cluster's node yet.** `deployment.yaml` uses `imagePullPolicy: Never`, meaning Kubernetes will only use an image that already exists on the node — it won't try to pull it. The image exists in your local Docker, but `kind` runs its nodes as separate containers with their own image store, so it needs to be explicitly loaded in with `kind load docker-image simple-app:v1 --name simple-app-cluster`.

6. What's left to do

1. Decide how ArgoCD should authenticate to the private repo — three options, roughly in order of simplicity:
 - **Make the repo public** and change `application.yaml`'s `repoURL` to the HTTPS form. No secrets to manage, but the code becomes publicly visible.
 - **Keep it private, use a GitHub Personal Access Token:** create a K8s Secret in the `argocd` namespace holding the token, register it as a repo credential, keep the HTTPS URL.
 - **Keep it private, use SSH:** create a K8s Secret from your existing deploy key (`~/.ssh/id_ed25519_omar_ait_bougnsa`) so `repo-server` can authenticate over SSH like your laptop does.
2. Add a `.gitignore` for `node_modules/`, then commit and push `service.yaml`, `namespace.yaml`, `kustomization.yaml` (and this report/README) to the repo — ArgoCD can only sync what's actually in git.
3. Load the image into the kind cluster: `kind load docker-image simple-app:v1 --name simple-app-cluster`.
4. Once the repo is reachable, ArgoCD's `syncPolicy.automated` (already set in `application.yaml`) will pick up the manifests automatically and create the `simple-app` namespace (via `CreateNamespace=true`), Deployment, and Service without any manual `kubectl apply`.
5. Verify with:

```
kubectl get application simple-app -n argocd
kubectl get pods,svc -n simple-app
```

Sync status should read `Synced / Healthy`, and the pods should show `Running`.